Spark Delta

In this train, we'll focus on the seamless integration and implementation of streaming data pipelines using Delta Live Tables (DLT) and Apache Spark.

Learning objectives

By the end of this train you should be able to:

- [Placeholder for objective 1]
- [Placeholder for objective 2]
- [Placeholder for objective 3]

Learning Objectives

- Gain the ability to design and implement streaming data pipelines using Delta Live Tables and Apache Spark, integrating Spark to enhance real-time data processing and analytics.
- Learn to effectively integrate Spark within DLT pipelines to perform data transformation, analysis, and other advanced operations.

Setting Up Spark With Delta Lake

Before working with Delta Lake, ensure you've installed the compatible version of Spark (e.g., Spark 3.x with Delta 3.3.1). Use one of the following shells based on your preferred language:

- **PySpark shell:**

```
  pyspark --packages io.delta:delta-spark_2.12:3.3.1 --conf
"spark.sql.extensions=io.delta.sql.DeltaSparkSessionExtension" --conf
"spark.sql.catalog.spark_catalog=org.apache.spark.sql.delta.catalog.DeltaCat
alog"
```

- **Spark SQL shell:**

```
bin/spark-sql --packages io.delta:delta-spark_2.12:3.3.1 \
--conf "spark.sql.extensions=io.delta.sql.DeltaSparkSessionExtension" \
--conf
"spark.sql.catalog.spark_catalog=org.apache.spark.sql.delta.catalog.DeltaCat
alog"
```

- **Python project setup:** Use pip and configure SparkSession:

```
  pip install delta-spark==3.3.1
```

```
  spark = configure_spark_with_delta_pip(builder).getOrCreate()
```

Working With Delta Tables

Once set up, you can begin creating and interacting with Delta tables:

- **Create:** Write DataFrames using `format("delta")`
- **Read:** Use `SELECT * FROM delta.`path`` or Spark DataFrame APIs
- **Update/Delete:** Use SQL-style commands like `UPDATE` or `DELETE`
- **Merge:** Perform upserts using `MERGE INTO`
- **Time travel:** Query old data with `VERSION AS OF`

Delta Lake provides transaction guarantees with its log structure and supports schema enforcement and evolution, ensuring consistency across operations.

Streaming With Delta Live Tables

Delta Live Tables (DLT) simplify the management of streaming pipelines with built-in observability and data quality features. You can write a streaming DataFrame to a

Delta table with the following code:

```
streamingDf = spark.readStream.format("rate").load()
stream = streamingDf.selectExpr("value as id") \
    .writeStream.format("delta") \
    .option("checkpointLocation", "/tmp/checkpoint") \
    .start("/tmp/delta-table")
```

Note: The example writes to `/tmp/delta-table`, which is suitable for local/demo environments.

Use `readStream` and `writeStream` to build both read and write pipelines. Delta ensures exactly-once processing even during concurrent reads/writes.

To read from the Delta table as a streaming source:

```
stream2 = spark.readStream.format("delta").load("/tmp/delta-table") \
    .writeStream.format("console").start()
```

Conclusion

Delta Lake combined with Apache Spark and DLT offers a powerful foundation for real-time, reliable, and scalable data processing. From ACID guarantees to time travel and structured streaming, the ecosystem simplifies building robust data pipelines for analytics and AI.

▼Key Takeaways ✖

- **Takeaway 1:** Delta Lake enhances Spark with ACID compliance, schema control, and streaming support.
- **Takeaway 2:** Setup requires compatible versions of Spark and Delta Lake (e.g., Delta 3.3.1).
- **Takeaway 3:** Delta tables can be queried, updated, deleted, and merged using SQL-like syntax.
- **Takeaway 4:** Delta Live Tables simplify streaming with checkpointing and consistent state management.
- **Takeaway 5:** Time travel in Delta enables querying older data versions with ease.